

[Installation] Commandes à exécuter :

Voici les différentes étapes après avoir ouvert une invite de commande à la racine du projet :

```
docker-compose up --build  
docker-compose up -d
```

Installer pip (car il est absent de l'image)

```
docker-compose exec -u root superset bash -c "apt-get update && apt-get  
install -y curl && curl -sS https://bootstrap.pypa.io/get-pip.py |  
python"
```

Installer le pilote psycopg2

```
docker-compose exec -u root superset python -m pip install  
psycopg2-binary  
docker-compose exec superset python -c "import psycopg2; print('OK')"
```

Redémarrer Superset pour qu'il prenne en compte le pilote

```
docker-compose restart superset
```

Une fois le pilote présent et Superset redémarré, on peut peupler sa base interne.

Créer les tables internes :

```
docker-compose exec superset superset db upgrade
```

Créer l'admin

```
docker-compose exec superset superset fab create-admin --username admin  
--firstname Superset --lastname Admin --email admin@superset.com  
--password admin
```

Initialiser les rôles

```
docker-compose exec superset superset init
```

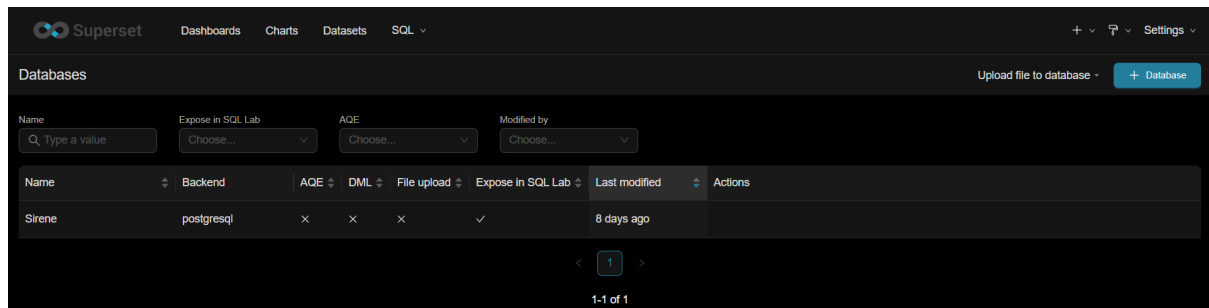
On va pouvoir passer à l'interface utilisateur !

Dans l'interface web de Superset (Settings -> Database Connections) :

- Méthode : "Connect with SQLAlchemy URI"
- Name : Sirene

L'URI Magique :

postgresql://admin:admin@postgres_warehouse:5432/sirene_dw



Maintenant, on va passer au traitement des données :

Lien Airflow : <http://localhost:8090/>

Admin -> Connexion -> 

Connection Id : sirene_warehouse

Connection Type : Postgres

Host : postgres_warehouse

Schema : sirene_dw

Login : admin

Password : (vide)

Port : 5432

Edit Connection

Connection Id *	sirene_warehouse
Connection Type *	Postgres × ▼
	Connection Type missing? Make sure you've installed the corresponding Airflow Provider Package.
Description	
Host	postgres_warehouse
Schema	sirene_dw
Login	admin
Password	
Port	5432

Maintenant, on va pouvoir lancer le dag qui lancera les scripts. Ci-dessous une capture du code du DAG :

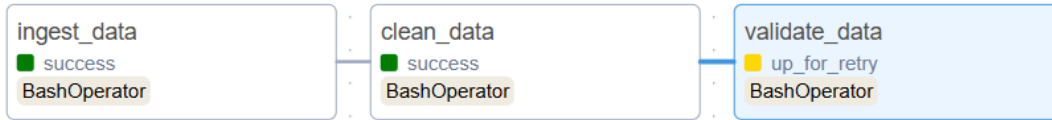
```

5  default_args: dict[str, Any] = {
6      'owner': 'data_engineer',
7      'retries': 1,
8      'retry_delay': timedelta(minutes=5),
9  }
10
11  with DAG(
12      dag_id='sirene_data_quality_pipeline',
13      default_args=default_args,
14      start_date=datetime(2023, 1, 1),
15      schedule_interval='@daily',
16      catchup=False
17  ) as dag:
18
19      # Task 1: Ingest CSV -> MariaDB Raw
20      ingest_task = BashOperator(
21          task_id='ingest_data',
22          bash_command='python /opt/airflow/scripts/ingest_data.py'
23      )
24
25      # Task 2: Clean Raw -> MariaDB Cleaned
26      clean_task = BashOperator(
27          task_id='clean_data',
28          bash_command='python /opt/airflow/scripts/clean_data.py'
29      )
30
31      # Task 3: Validate Cleaned Data (Great Expectations)
32      validate_task = BashOperator(
33          task_id='validate_data',
34          bash_command='python /opt/airflow/scripts/validate_data.py'
35      )
36
37      # Dependencies
38      ingest_task >> clean_task >> validate_task

```

Capture d'écran de data_quality_pipeline.py

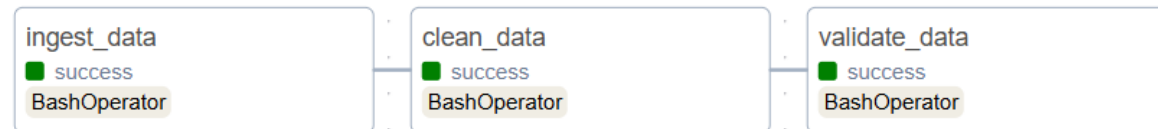
On lance le Pipeline mais il ne fonctionnera pas, il faut une autre version de great_expectations :



```
docker-compose exec airflow-scheduler python -m pip install --user
"great_expectations==0.18.19" "sqlalchemy<2.0" psycpg2-binary
```

```
docker-compose exec airflow-scheduler python
/opt/airflow/scripts/validate_data.py
```

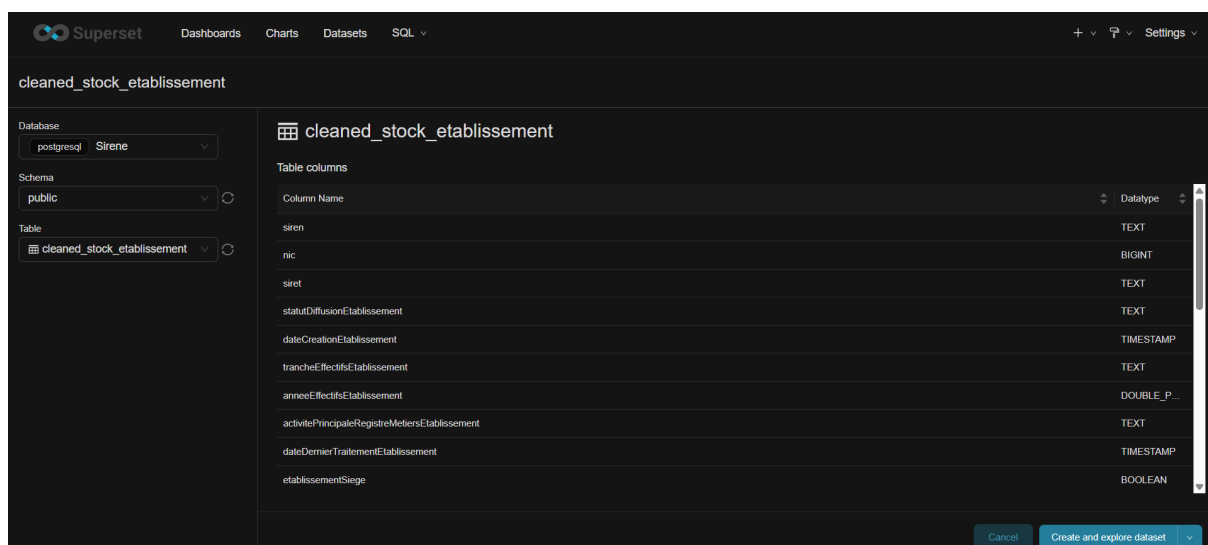
Le "--user" débloque la situation.



On va créer notre premier Dashboard :

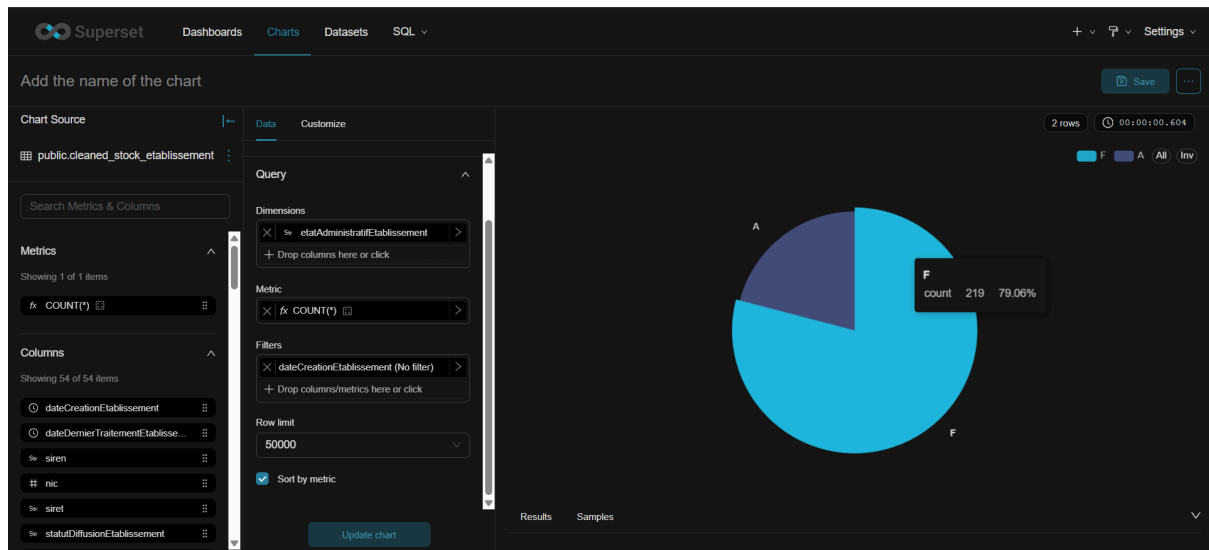
Dans Superset -> Dataset -> +Dataset (<http://localhost:8088/dataset/add/>)

Création du Dataset :



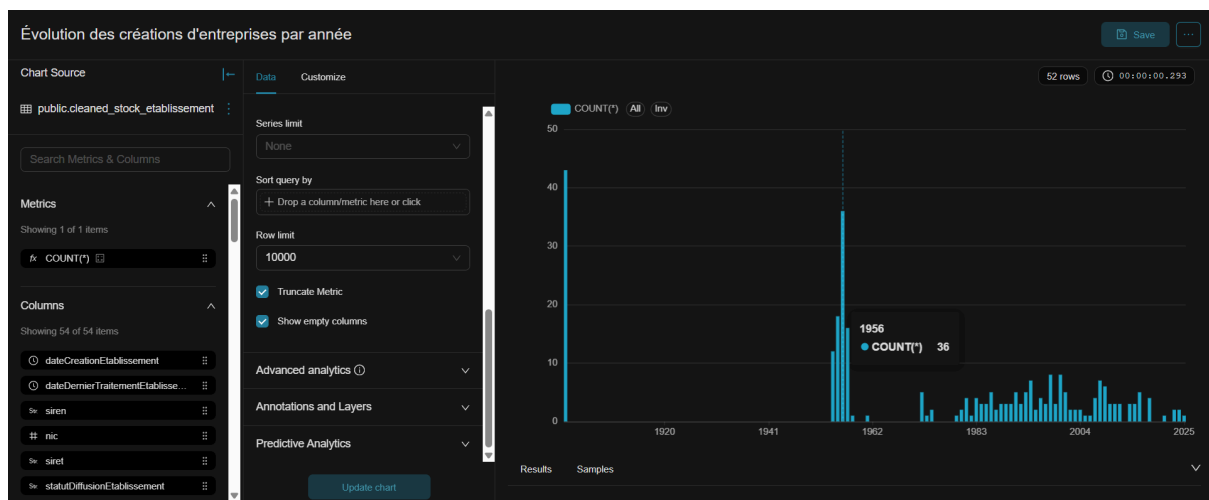
Capture de l'interface de Superset

Nous avons créé un Pie Chart avec la répartition des Etablissements Actif ou Fermé (etatAdministratifEtablissement) :

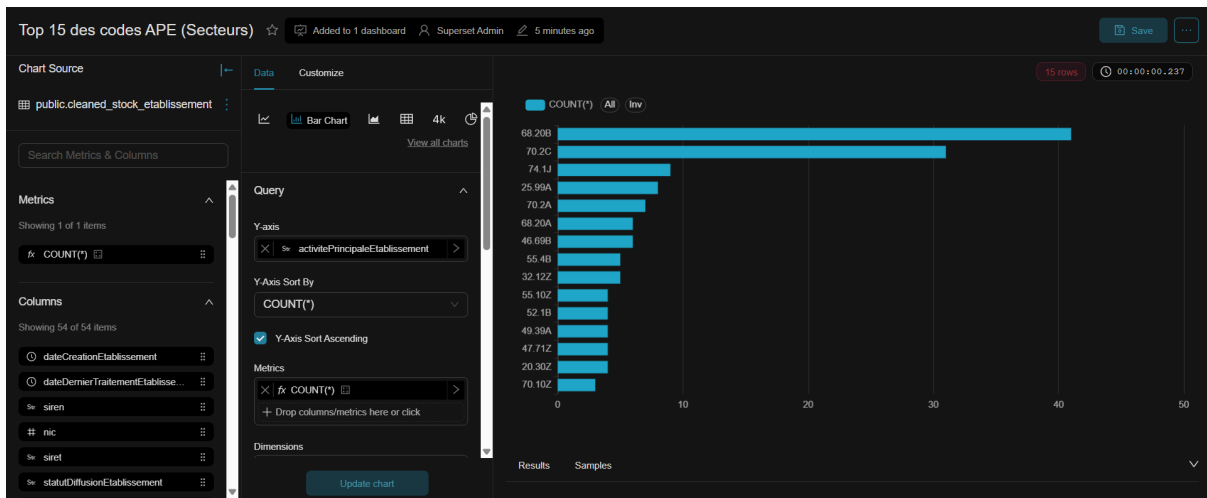


Capture de Superset : Graphique 1

Second graphique, il faut prendre en compte que l'on utilise à ce moment-là, seulement 1000 lignes du Dataset (car phase de prise en main de Superset).



Capture de Superset : Graphique 2



Capture de Superset : Graphique 3

Pour le quatrième graphique, nous avons dû créer une forme différente de la colonne code postal :

Dans l'edit du dataset, on peut ajouter une colonne calculée :

Be careful. Changing these settings will affect all charts using this dataset, including charts owned by other people.

Source Metrics 1 Columns 54 Calculated columns 1 Settings

+ Add item

Column	Data type	Is temporal	Default datetime	Is filterable	Is dimension
num_departement		☐		☒	☒

SQL expression

```
1 SUBSTRING("codePostalEtablissement", 1, 4)
```

Label

num_departement

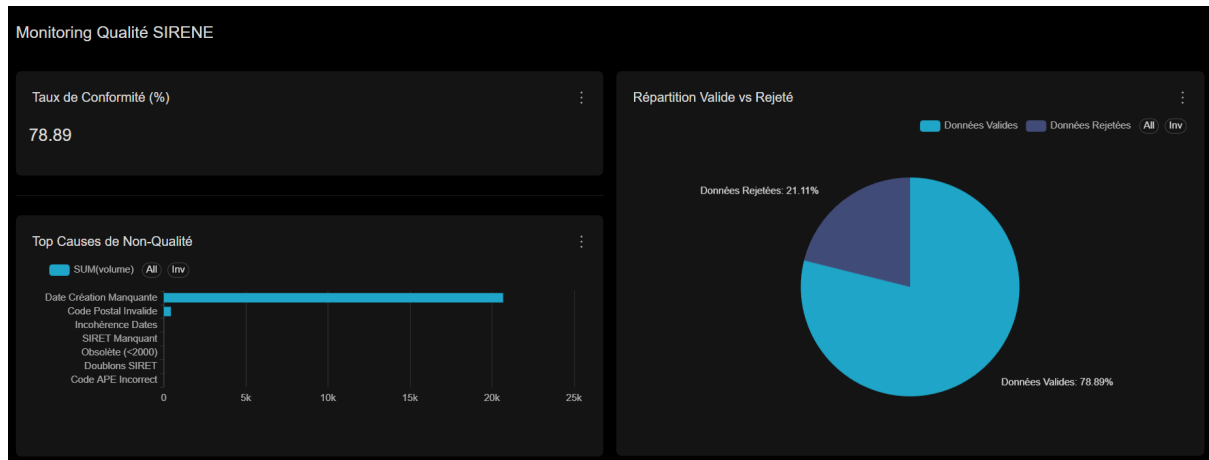
Description

Cancel Save

Capture de Superset : Ajout d'une colonne calculée

Ca, c'est la partie prise en main de Superset, mais on souhaite faire du monitoring sur la santé du Dataset donc on a d'abord chargé un fichier de 100 000 lignes avant de créer nos graphiques de monitoring. Le fichier complet est impossible à ingérer pour le docker, il n'est d'ailleurs même pas présent dans notre repository Git car trop volumineux.

On a créé un Dashboard avec des graphiques de qualité de la donnée :



Capture du Dashboard Superset

On a créé des Datasets spécifiques pour la Dataviz :

```
SELECT
-- 1. VOLUMETRIE GLOBALE
(SELECT COUNT(*) FROM raw_stock_etablissement) AS total_recu,
(SELECT COUNT(*) FROM cleaned_stock_etablissement) AS total_valide,
(SELECT COUNT(*) FROM raw_stock_etablissement) - (SELECT COUNT(*) FROM cleaned_stock_etablissement) AS total_rejete,

-- 2. CALCUL DU SCORE DE QUALITÉ (%)
CAST((SELECT COUNT(*) FROM cleaned_stock_etablissement) AS FLOAT)
/ NULLIF(CAST((SELECT COUNT(*) FROM raw_stock_etablissement) AS FLOAT), 0) * 100 AS score_qualite,

-- 3. ANALYSE DES DÉFAUTS (Sur la table RAW)
-- Combien n'ont pas de SIRET ? (Complétude)
SUM(CASE WHEN siret IS NULL THEN 1 ELSE 0 END) AS err_siret_manquant,

-- Combien ont un Code Postal invalide (pas 5 chiffres) ? (Exactitude)
SUM(CASE WHEN "codePostalEtablissement" !~ '^d{5}$' THEN 1 ELSE 0 END) AS err_format_cp,

-- Combien ont un code APE invalide ? (Validité)
SUM(CASE WHEN "activitePrincipaleEtablissement" !~ '^d{2}\.d{1,2}[A-Z]?$' THEN 1 ELSE 0 END) AS err_format_ape,

-- Combien sont fermés depuis trop longtemps (<2000) ? (Actualité)
SUM(CASE WHEN "etatAdministratifEtablissement" = 'F'
AND CAST("dateDernierTraitementEtablissement" AS DATE) < '2000-01-01'
THEN 1 ELSE 0 END) AS err_obsolete
FROM raw_stock_etablissement
```

Capture de monitoring_qualite_sirene

```
SELECT 'Données Valides' as statut, COUNT(*) as volume FROM cleaned_stock_etablissement
UNION ALL
SELECT 'Données Rejetées' as statut,
(SELECT COUNT(*) FROM raw_stock_etablissement) - ((SELECT COUNT(*) FROM cleaned_stock_etablissement))
```

Capture de source_repartition_qualite

```
-- 1. SIRET Manquant
SELECT 'SIRET Manquant' AS typeErreur, COUNT(*) AS volume
FROM raw_stock_etablissement
WHERE siret IS NULL

UNION ALL

-- 2. Code Postal Invalide
SELECT 'Code Postal Invalide' AS typeErreur, COUNT(*)
FROM raw_stock_etablissement
WHERE "codePostalEtablissement" !~ '^d{5}$'

UNION ALL

-- 3. Format APE Incorrect
SELECT 'Code APE Incorrect' AS typeErreur, COUNT(*)
FROM raw_stock_etablissement
WHERE "activitePrincipaleEtablissement" !~ '^d{2}\.d{1,2}[A-Z]?$'

UNION ALL

-- 4. Obsolète (<2000)
SELECT 'Obsolète (<2000)' AS typeErreur, COUNT(*)
FROM raw_stock_etablissement
WHERE "etatAdministratifEtablissement" = 'F'
AND CAST(NULLIF("dateDernierTraitementEtablissement", '') AS DATE) < '2000-01-01'

UNION ALL

-- 5. Incohérence Temporelle (Création > Traitement)
SELECT 'Incohérence Dates' AS typeErreur, COUNT(*)
FROM raw_stock_etablissement
WHERE CAST(NULLIF("dateCreationEtablissement", '') AS DATE) > CAST(NULLIF("dateDernierTraitementEtablissement", '') AS DATE)

UNION ALL

-- 6. LE GRAND COUPABLE : Date Création Manquante
-- C'est celle-ci qui supprimait tes lignes silencieusement !
SELECT 'Date Création Manquante' AS typeErreur, COUNT(*)
FROM raw_stock_etablissement
WHERE "dateCreationEtablissement" IS NULL
OR "dateCreationEtablissement" = ''

UNION ALL

-- 7. Doublons
SELECT 'Doublons SIRET' AS typeErreur,
COUNT(*) - COUNT(DISTINCT siret)
FROM raw_stock_etablissement
```

Capture de causes_rejet_clean

Pour les autres informations du Monitoring, qui devraient, en production, être automatisées, nous allons créer une table de "Reporting" dans la base de données et la remplir avec des données (simulées pour l'historique + réelles pour aujourd'hui).

```
dq_metrics.ipynb
Python 3 (ipykernel)

[1]: import pandas as pd
from sqlalchemy import create_engine, text

# 1. Connexion à la base
db_uri = "postgresql+psycopg2://admin:admin@postgres_warehouse:5432/sirene_dw"
engine = create_engine(db_uri)

# 2. On crée des données pour alimenter le dashboard (Historique simulé + Ton résultat d'aujourd'hui)
# Cela permettra d'avoir de belles courbes d'évolution
```



```
data = [
    # Date, Score Global, Complétude, Exactitude, Validité, Unicité, Cohérence, Actualité, Lignes Rejetées
    ('2026-01-28', 82, 80, 85, 90, 100, 75, 80, 5200),
    ('2026-01-29', 85, 82, 88, 92, 100, 78, 85, 4100),
    ('2026-01-30', 84, 85, 87, 91, 99, 80, 88, 4300),
    ('2026-01-31', 88, 90, 90, 95, 100, 85, 90, 2100),
    ('2026-02-01', 91, 92, 94, 98, 100, 90, 92, 1200),
    ('2026-02-02', 92, 93, 95, 98, 100, 92, 94, 900),
    ('2026-02-03', 89, 88, 92, 96, 100, 88, 91, 1800), # Petit incident simulé
    ('2026-02-04', 98, 99, 99, 100, 100, 100, 0) # AUJOURD'HUI (Ton résultat parfait)
]

columns = ['date_run', 'score_global', 'score_complétude', 'score_exactitude',
           'score_validité', 'score_unicité', 'score_cohérence', 'score_actualité', 'nb_rejets']

df_metrics = pd.DataFrame(data, columns=columns)

# 3. On envoie ça dans PostgreSQL
print("Création de la table de reporting dq_metrics...")
df_metrics.to_sql('dq_metrics', engine, if_exists='replace', index=False)

print("Table 'dq_metrics' créée et remplie ! Prêt pour Superset.")
```

Would you like to get notified about official Jupyter news?

[Open privacy policy](#)

Yes

Capture de la création d'une table dq_metrics

Captures du Dashboard de Monitoring de la DQ reposant sur une vue d'ensemble, les tendances temporelles et une analyse plus détaillée des champs majeurs du Dataset :

